Name: Rahul Gupta

Reg No: 2547242

Class: 4 MCA B

Lab 2

Aim: Create and configure an AWS S3 Bucket using Python

Code:

app.py

```
import streamlit as st
import boto3
from botocore.exceptions import ClientError, NoCredentialsError

st.set_page_config(page_title="S3 Manager")
st.title("Amazon S3 Manager")

# Sidebar - AWS Credentials
st.sidebar.header("AWS Credentials")
aws_access_key = st.sidebar.text_input("Access Key ID", type="password")
aws_secret_key = st.sidebar.text_input("Secret Access Key", type="password")
aws_session_token = st.sidebar.text_input("Session Token", type="password")

# Connection state
if "connected" not in st.session_state:
   st.session_state.connected = False

if st.sidebar.button("Connect to AWS"):
   if not (aws_access_key and aws_secret_key and aws_session_token):
       st.sidebar.error("Please fill in all three credential fields.")
       st.session_state.connected = False
   else:
       try:
           test_client = boto3.client(
               "s3",
               aws_access_key_id=aws_access_key,
               aws_secret_access_key=aws_secret_key,
               aws_session_token=aws_session_token,
           )
           test_client.list_buckets()
           st.session_state.connected = True
       except (ClientError, NoCredentialsError):
```

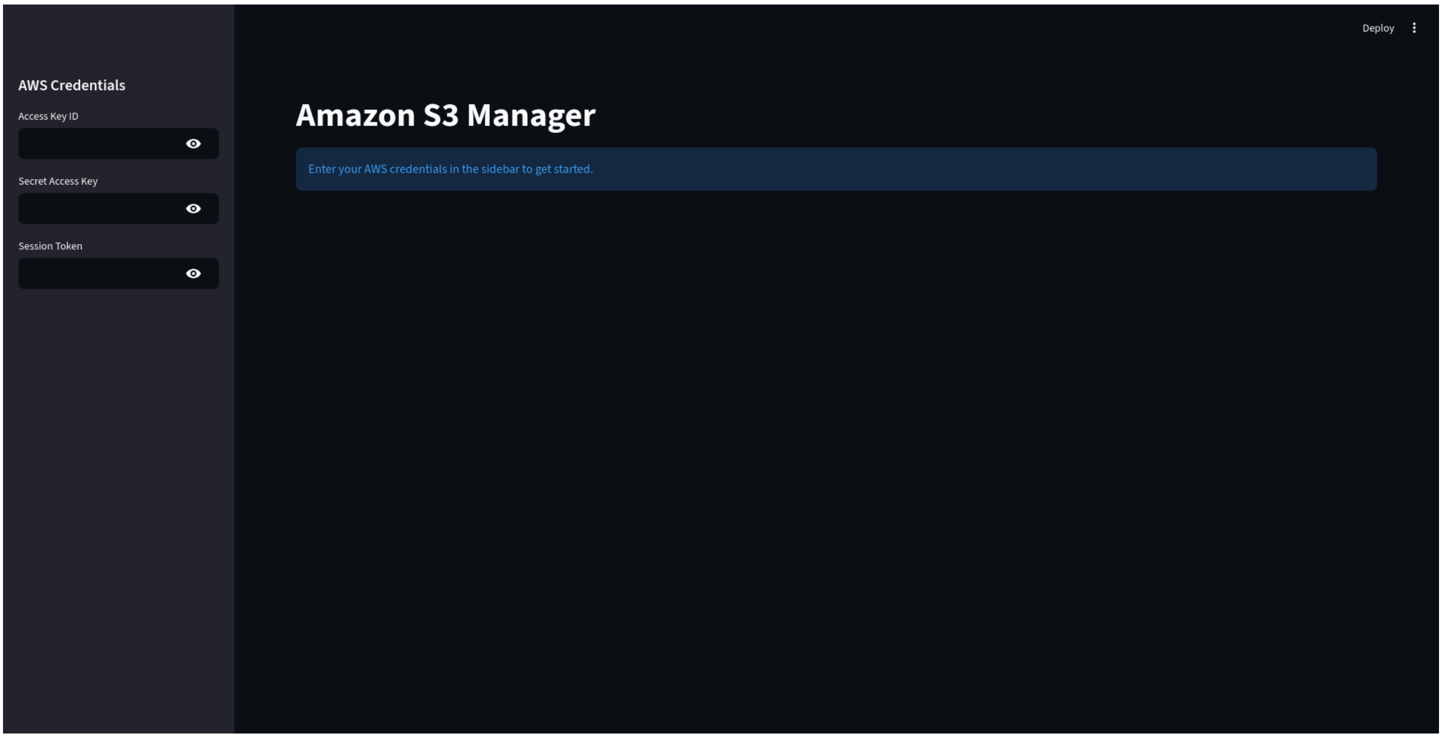
```python
            st.session_state.connected = False
            st.sidebar.error("Connection failed. Check your credentials.")

# Show connection status
if st.session_state.connected:
    st.sidebar.success("Status: Connected")
else:
    st.sidebar.warning("Status: Not Connected")

def get_client():
    return boto3.client(
        "s3",
        aws_access_key_id=aws_access_key or None,
        aws_secret_access_key=aws_secret_key or None,
        aws_session_token=aws_session_token or None,
    )

if not st.session_state.connected:
    st.info("Connect to AWS using the sidebar to get started.")
    st.stop()

tab1, tab2, tab3 = st.tabs(["Create Bucket", "Upload Files", "Change Access"])

# Tab 1 - Create Bucket
with tab1:
    st.header("Create a New S3 Bucket")
    bucket_name = st.text_input("Bucket Name")
    region = st.selectbox("Region", [
        "us-east-1", "us-east-2", "us-west-1", "us-west-2",
        "ap-south-1", "eu-west-1", "eu-central-1"
    ])

    if st.button("Create Bucket"):
        if not bucket_name:
            st.error("Please enter a bucket name.")
        else:
            try:
                s3 = get_client()
                if region == "us-east-1":
                    s3.create_bucket(Bucket=bucket_name)
                else:
                    s3.create_bucket(
                        Bucket=bucket_name,
                        CreateBucketConfiguration={"LocationConstraint": region}
                    )
                st.success(f"Bucket '{bucket_name}' created in '{region}'!")
            except ClientError as e:
                st.error(e.response["Error"]["Message"])
```

```python
# Tab 2 - Upload Files
with tab2:
   st.header("Upload Files to a Bucket")
   try:
       s3 = get_client()
       all_buckets = [b["Name"] for b in s3.list_buckets().get("Buckets", [])]
   except Exception:
       all_buckets = []

   upload_bucket = st.selectbox("Select Bucket", all_buckets or ["No buckets found"],
key="upload_bucket")
   uploaded_files = st.file_uploader("Choose files", accept_multiple_files=True)

   if st.button("Upload Files"):
       if not uploaded_files:
           st.error("Please select at least one file.")
       else:
           s3 = get_client()
           for file in uploaded_files:
               try:
                   s3.upload_fileobj(file, upload_bucket, file.name)
                   st.success(f"Uploaded '{file.name}' to '{upload_bucket}'")
               except ClientError as e:
                   st.error(f"{file.name}: {e.response['Error']['Message']}")

# Tab 3 - Change Access
with tab3:
   st.header("Change Object Access Level")
   try:
       s3 = get_client()
       all_buckets = [b["Name"] for b in s3.list_buckets().get("Buckets", [])]
   except Exception:
       all_buckets = []

   acl_bucket = st.selectbox("Select Bucket", all_buckets or ["No buckets found"],
key="acl_bucket")

   objects = []
   if acl_bucket and acl_bucket != "No buckets found":
       try:
           s3 = get_client()
           resp = s3.list_objects_v2(Bucket=acl_bucket)
           objects = [o["Key"] for o in resp.get("Contents", [])]
       except Exception:
           objects = []

   selected_object = st.selectbox("Select File", objects or ["No files found"])
```

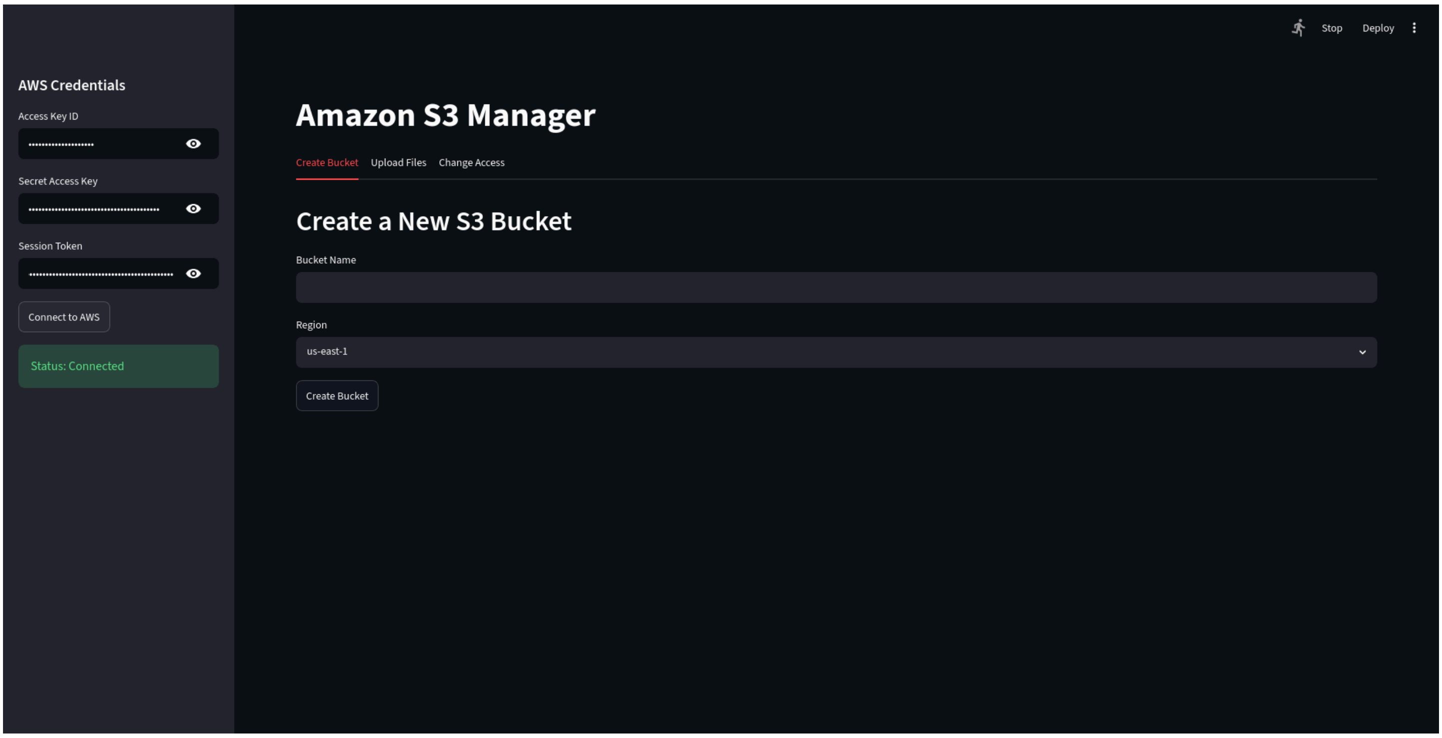
```
    acl_choice = st.selectbox("Access Level", [
        "private", "public-read", "public-read-write", "authenticated-read"
    ])

    if st.button("Apply Access Level"):
        if not objects:
            st.error("No file selected.")
        else:
            try:
                s3 = get_client()
                s3.put_object_acl(Bucket=acl_bucket, Key=selected_object,
ACL=acl_choice)
                st.success(f"Access for '{selected_object}' changed to
'{acl_choice}'.")

                acl_resp = s3.get_object_acl(Bucket=acl_bucket, Key=selected_object)
                st.subheader("Current Grants")
                for grant in acl_resp.get("Grants", []):
                    grantee = grant["Grantee"]
                    name = grantee.get("DisplayName") or grantee.get("URI", "Unknown")
                    st.write(f"{name} => {grant['Permission']}")

            except ClientError as e:
                err_code = e.response["Error"]["Code"]
                if err_code == "AccessControlListNotSupported":
                    st.warning("This bucket has Block Public Access or Object
Ownership settings that prevent ACL changes. Disable them in the AWS Console first.")
                else:
                    st.error(e.response["Error"]["Message"])
```

Output:

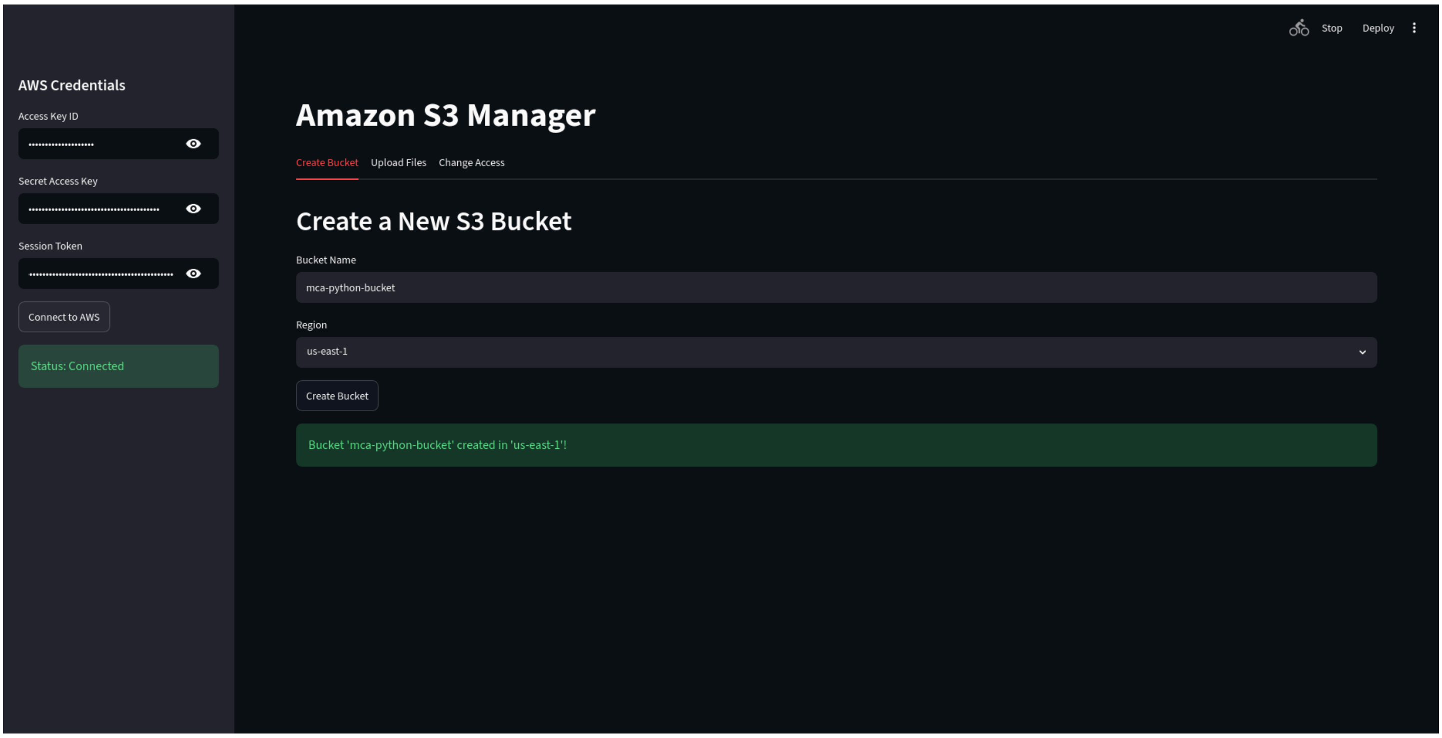
Streamlit Website UI

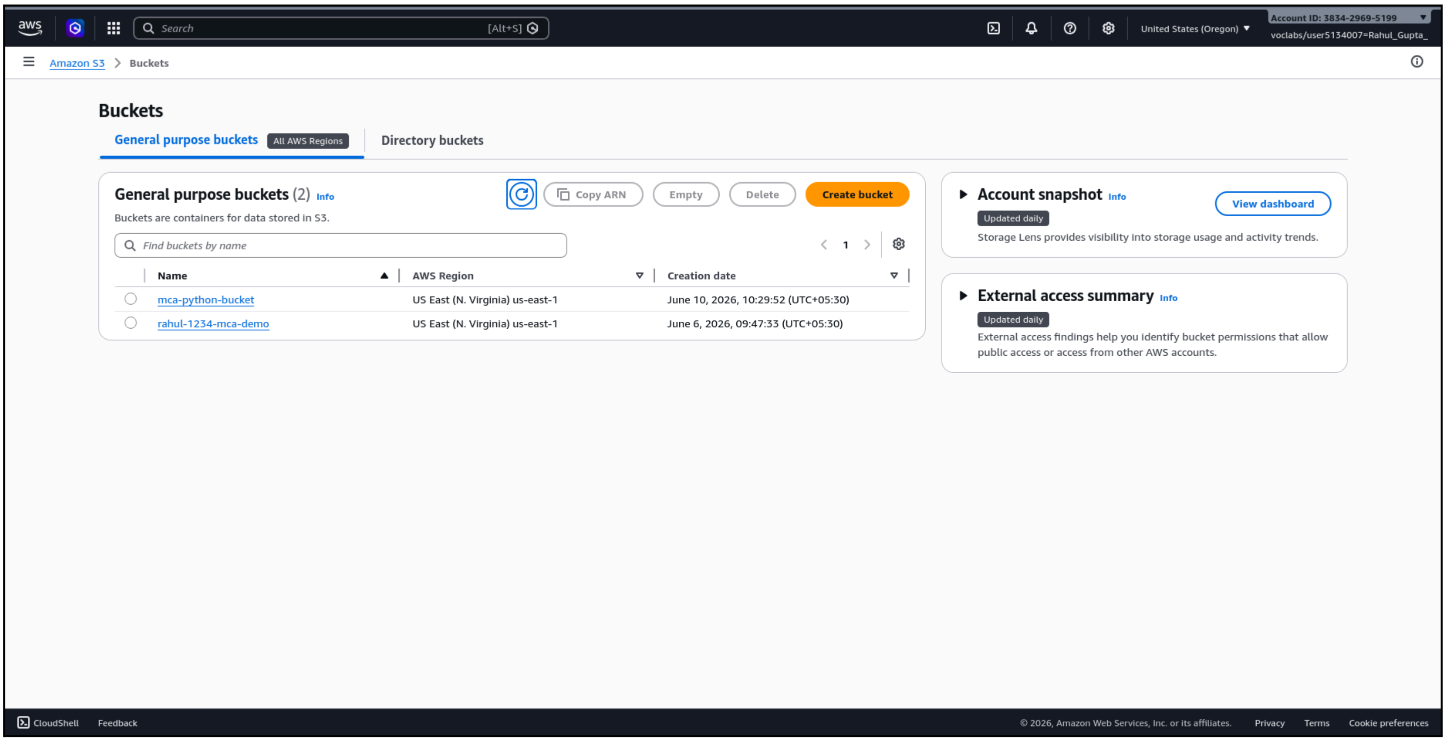


Configure AWS Credentials

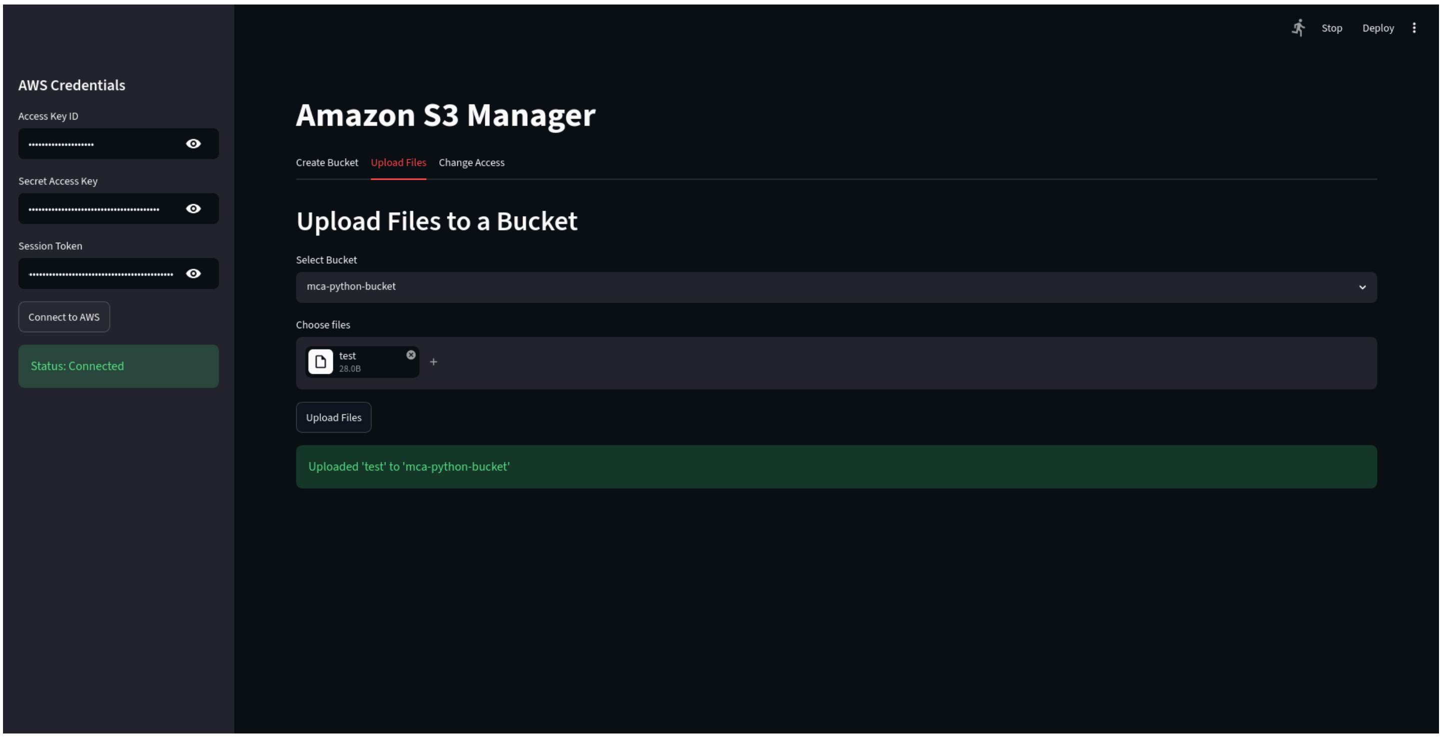


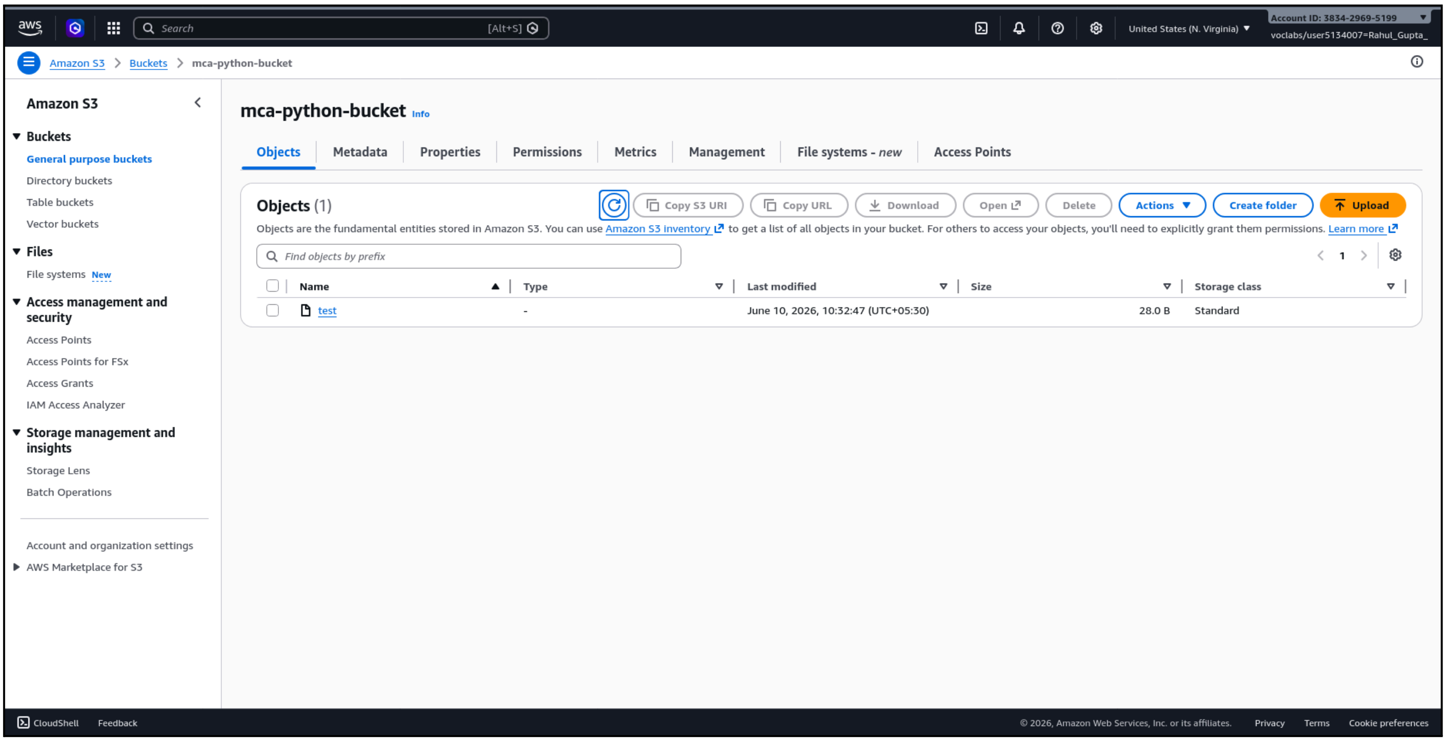
Bucket Creation





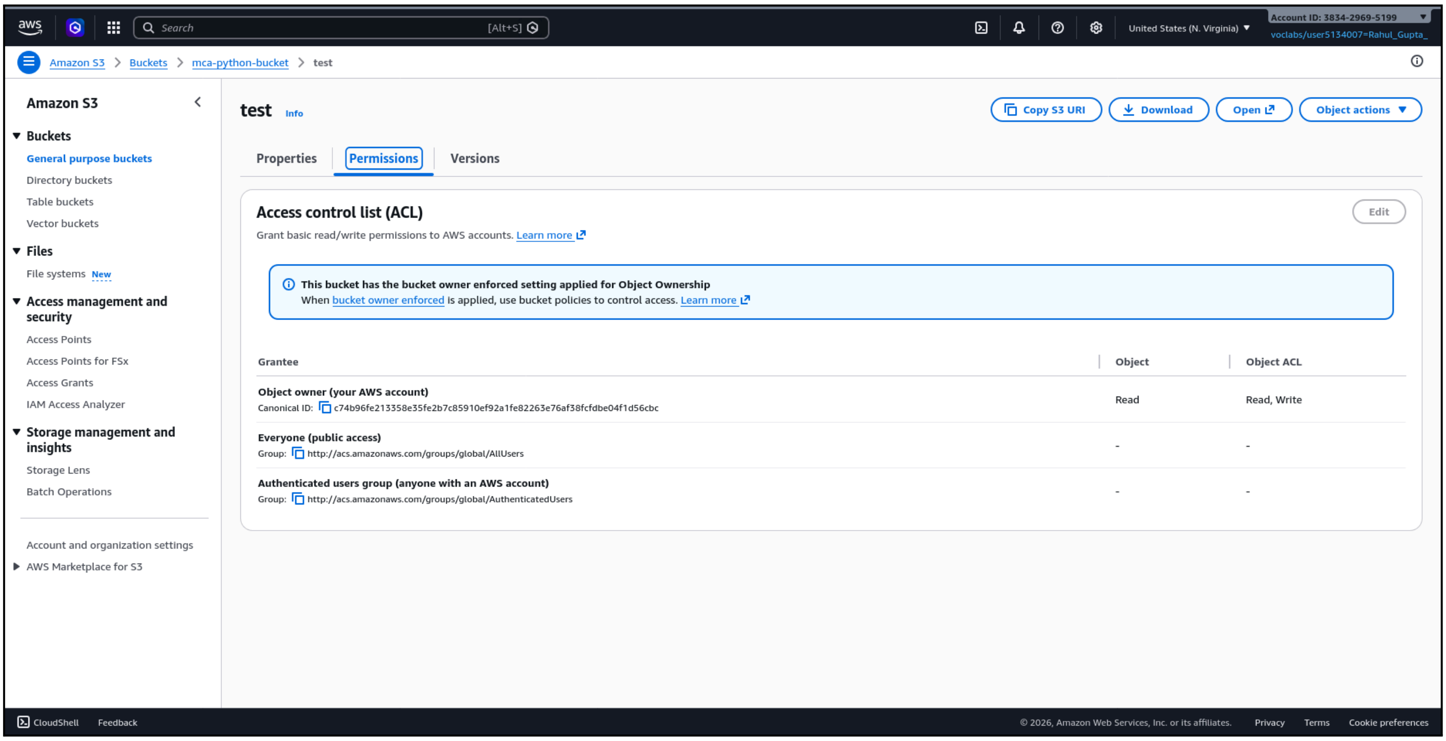
Upload File

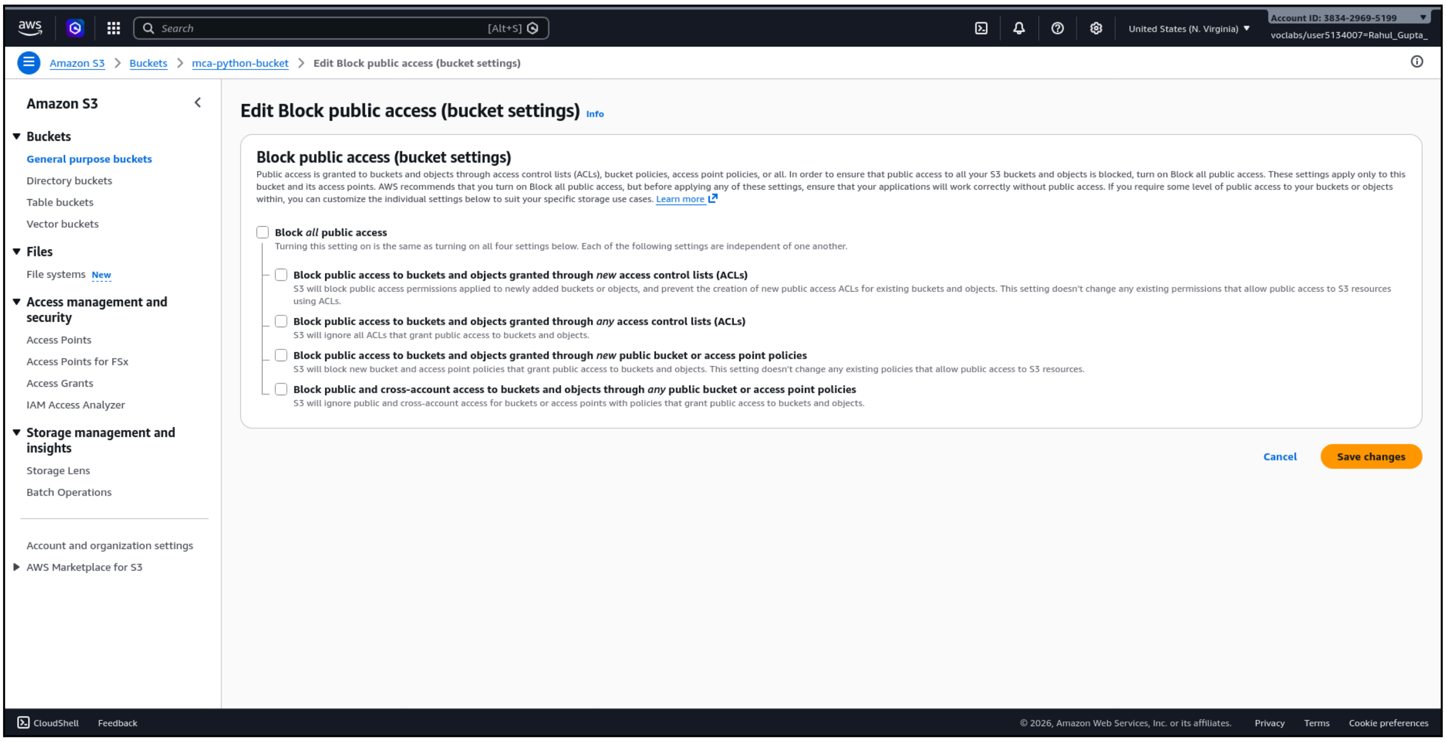

Stop
Deploy
AWS Credentials
Access Key ID
Secret Access Key
Session Token
Connect to AWS
Status: Connected
Amazon S3 Manager
Create Bucket
Upload Files
Change Access
Upload Files to a Bucket
Select Bucket
mca-python-bucket
Choose files
test
28.0B
Upload Files
Uploaded 'test' to 'mca-python-bucket'

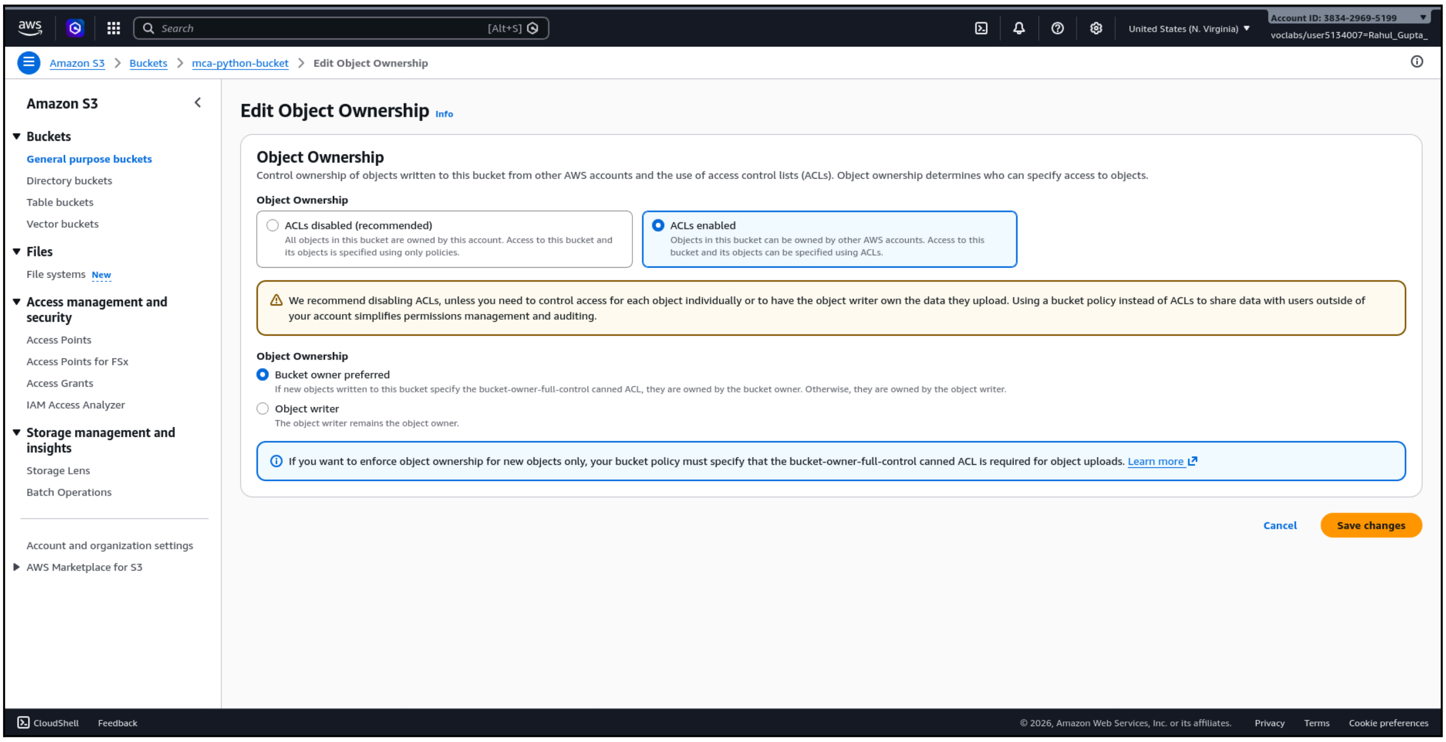

Search
[Alt+S]
United States (N. Virginia)
Account ID: 3834-2969-5199
voclabs/user5134007=Rahul_Gupta_
Amazon S3 > Buckets > mca-python-bucket
Amazon S3
Buckets
General purpose buckets
Directory buckets
Table buckets
Vector buckets
Files
File systems New
Access management and security
Access Points
Access Points for FSx
Access Grants
IAM Access Analyzer
Storage management and insights
Storage Lens
Batch Operations
Account and organization settings
AWS Marketplace for S3
mca-python-bucket Info
Objects
Metadata
Properties
Permissions
Metrics
Management
File systems - new
Access Points
Objects (1)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more
Find objects by prefix
Name
Type
Last modified
Size
Storage class
test
-
June 10, 2026, 10:32:47 (UTC+05:30)
28.0 B
Standard
CloudShell
Feedback
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

ACL Change of Object

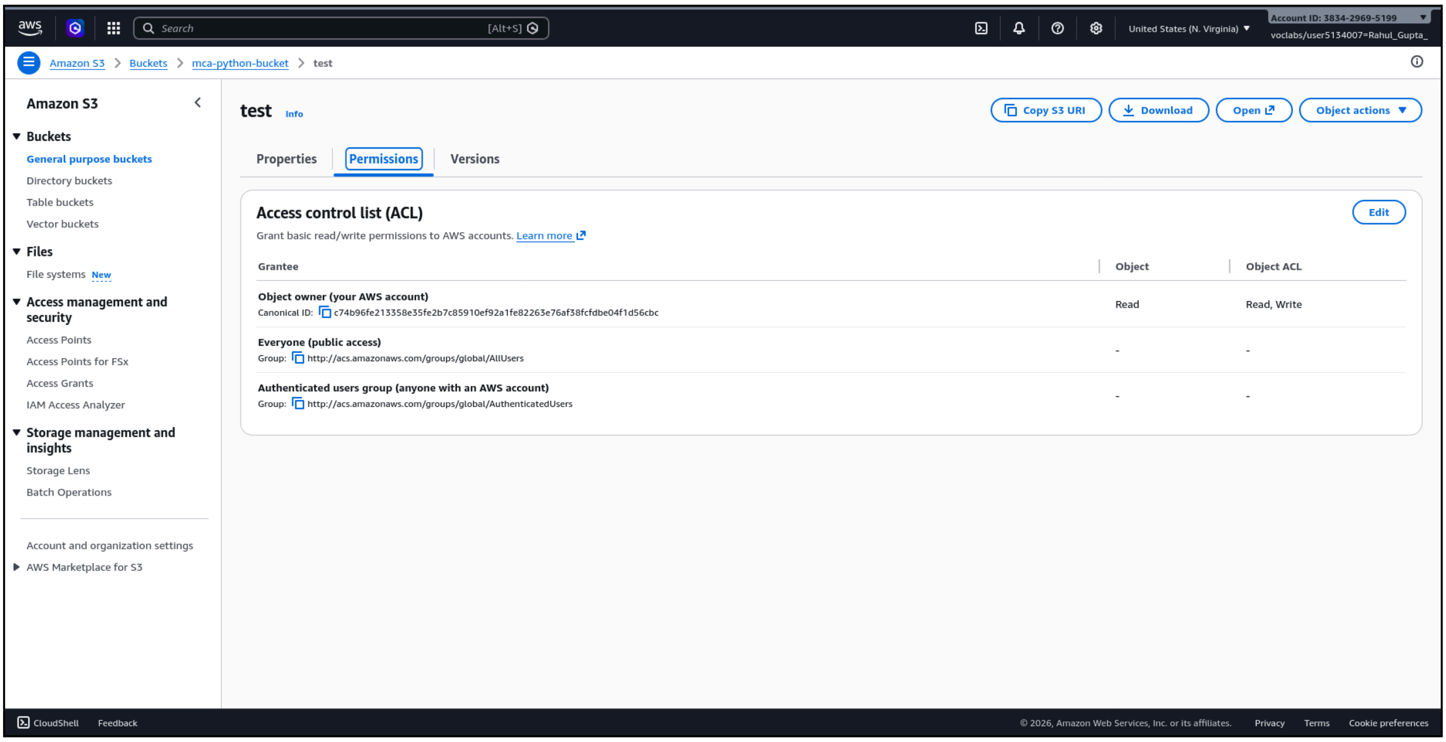
(the - means, acl is private)



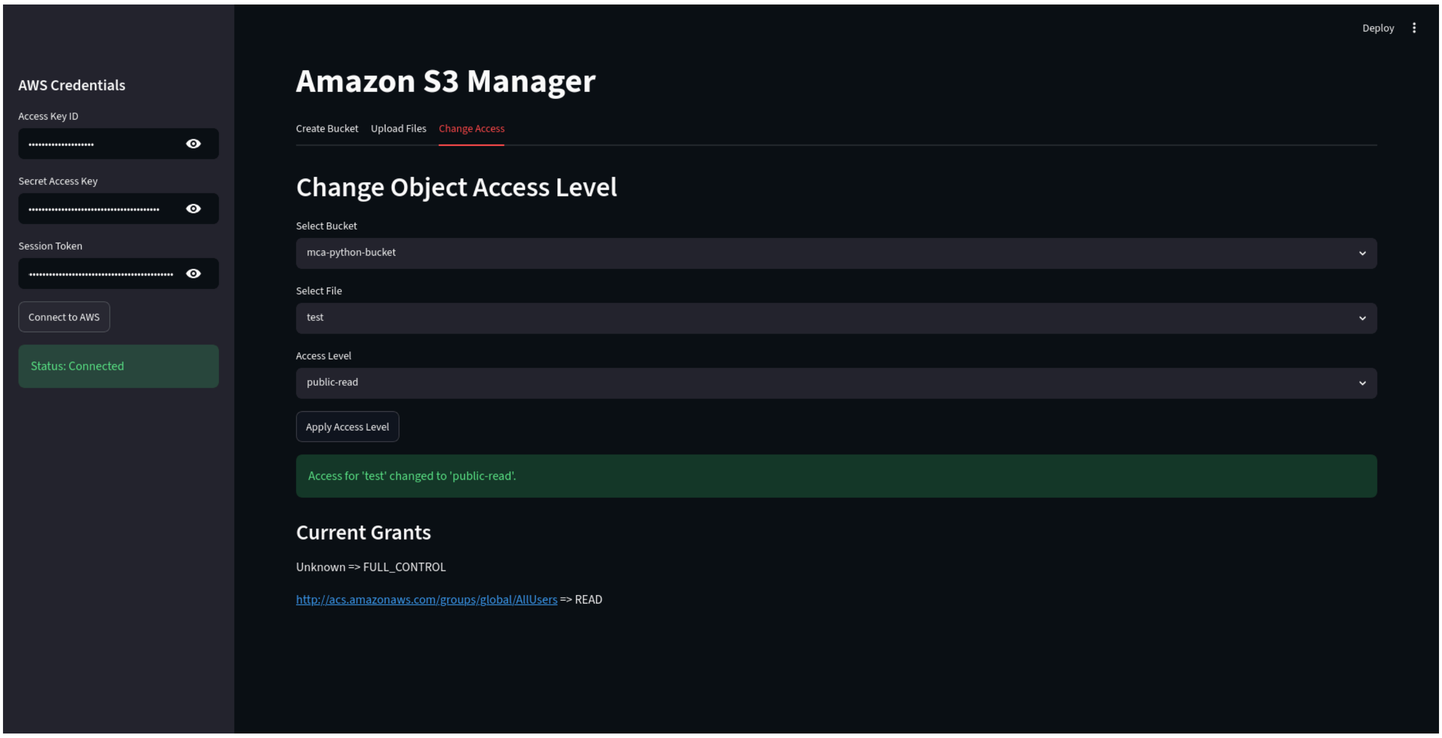


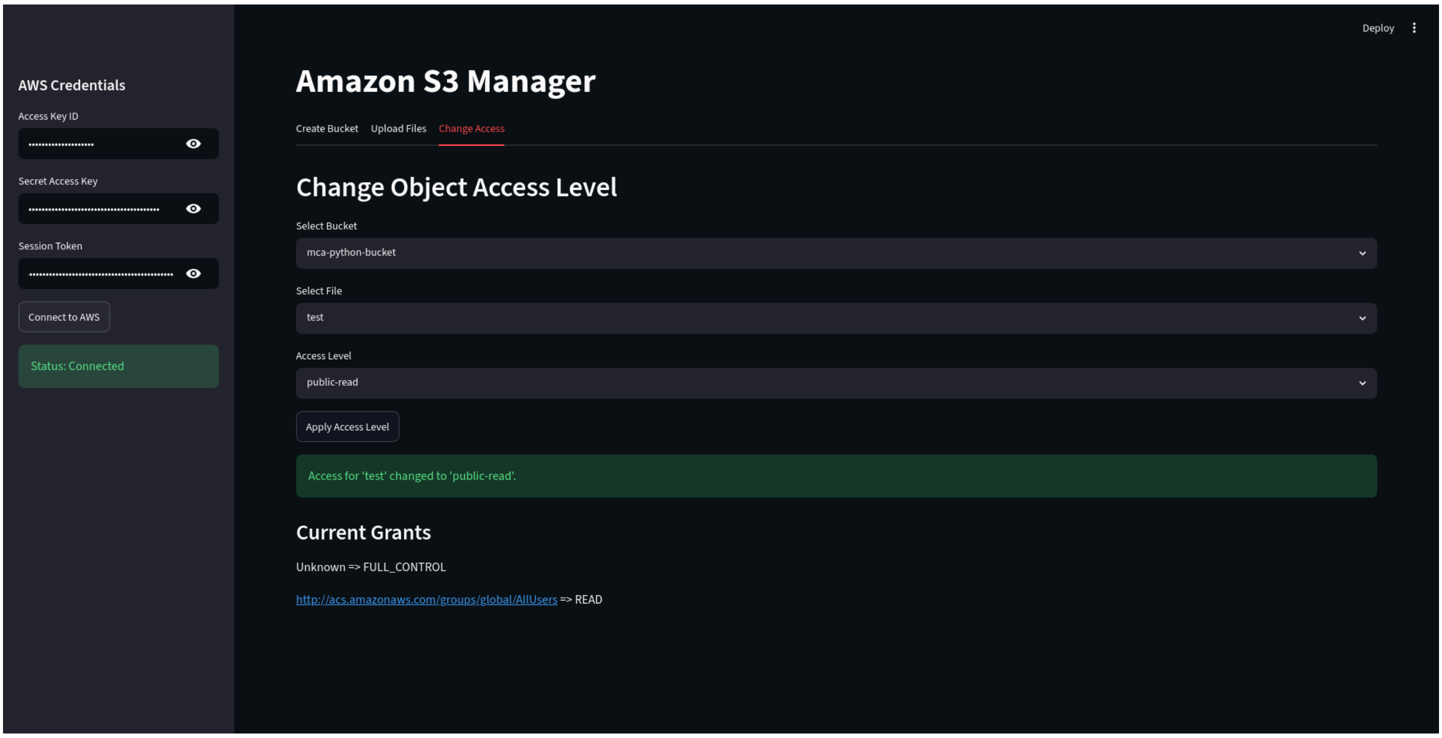


Search [Alt+S]
United States (N. Virginia)
Account ID: 3834-2969-5199
voclabs/user5134007=Rahul_Gupta_
Amazon S3 > Buckets > mca-python-bucket > Edit Object Ownership
Amazon S3
Buckets
General purpose buckets
Directory buckets
Table buckets
Vector buckets
Files
File systems New
Access management and security
Access Points
Access Points for FSx
Access Grants
IAM Access Analyzer
Storage management and insights
Storage Lens
Batch Operations
Account and organization settings
AWS Marketplace for S3
Edit Object Ownership Info
Object Ownership
Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.
Object Ownership
ACLs disabled (recommended)
All objects in this bucket are owned by this account. Access to this bucket and its objects is specified using only policies.
ACLs enabled
Objects in this bucket can be owned by other AWS accounts. Access to this bucket and its objects can be specified using ACLs.
We recommend disabling ACLs, unless you need to control access for each object individually or to have the object writer own the data they upload. Using a bucket policy instead of ACLs to share data with users outside of your account simplifies permissions management and auditing.
Object Ownership
Bucket owner preferred
If new objects written to this bucket specify the bucket-owner-full-control canned ACL, they are owned by the bucket owner. Otherwise, they are owned by the object writer.
Object writer
The object writer remains the object owner.
If you want to enforce object ownership for new objects only, your bucket policy must specify that the bucket-owner-full-control canned ACL is required for object uploads. Learn more
Cancel
Save changes
CloudShell
Feedback
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences



Search [Alt+S]
United States (N. Virginia)
Account ID: 3834-2969-5199
voclabs/user5134007=Rahul_Gupta_
Amazon S3 > Buckets > mca-python-bucket > test
Amazon S3
Buckets
General purpose buckets
Directory buckets
Table buckets
Vector buckets
Files
File systems New
Access management and security
Access Points
Access Points for FSx
Access Grants
IAM Access Analyzer
Storage management and insights
Storage Lens
Batch Operations
Account and organization settings
AWS Marketplace for S3
test Info
Copy S3 URI
Download
Open
Object actions
Properties
Permissions
Versions
Access control list (ACL)
Edit
Grant basic read/write permissions to AWS accounts. Learn more
Grantee
Object
Object ACL
Object owner (your AWS account)
Canonical ID: c74b96fe213358e35fe2b7c85910ef92a1fe82263e76af38fcfdbe04f1d56cbc
Read
Read, Write
Everyone (public access)
Group: http://acs.amazonaws.com/groups/global/AllUsers
-
-
Authenticated users group (anyone with an AWS account)
Group: http://acs.amazonaws.com/groups/global/AuthenticatedUsers
-
-
CloudShell
Feedback
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Deploy
AWS Credentials
Access Key ID
Secret Access Key
Session Token
Connect to AWS
Status: Connected
Amazon S3 Manager
Create Bucket
Upload Files
Change Access
Change Object Access Level
Select Bucket
mca-python-bucket
Select File
test
Access Level
public-read
Apply Access Level
Access for 'test' changed to 'public-read'.
Current Grants
Unknown => FULL_CONTROL
http://acs.amazonaws.com/groups/global/AllUsers => READ


Deploy
AWS Credentials
Access Key ID
Secret Access Key
Session Token
Connect to AWS
Status: Connected
Amazon S3 Manager
Create Bucket
Upload Files
Change Access
Change Object Access Level
Select Bucket
mca-python-bucket
Select File
test
Access Level
public-read
Apply Access Level
Access for 'test' changed to 'public-read'.
Current Grants
Unknown => FULL_CONTROL
http://acs.amazonaws.com/groups/global/AllUsers => READ